

PROJECT REPORT

DISASTER RESPONSE & RESCUE

NAVIGATION SYSTEM (RESCUENAV)

COMPUTER SCIENCE AND ENGINEERING

Submitted By:

Parasa Bhavyaswi (12502256)

Pudi Sai Charan Teja (12507339)

Ganta Gangadhar (12523695)

Kola Midhun Kumar (12524467)

Section: 9PV33

Under the Guidance of:

Deepak Kumar



L OVELY
P ROFESSIONAL
U NIVERSITY

School of Computing & Artificial Intelligence

Lovely Professional University, Phagwara

Academic Session: 2026-2027

DECLARATION

We hereby declare that the project report entitled "**Disaster Response & Rescue Navigation System**" submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering at Lovely Professional University, Phagwara, is an authentic record of our own work carried out under the guidance of Mr. Deepak Kumar. The matter embodied in this report has not been submitted by us for the award of any other degree or diploma.

(Signatures of the Candidates)

Parasa Bhavyaswi

Pudi Sai Charan Teja

Ganta Gangadhar

Kola Midhun Kumar

CERTIFICATE

This is to certify that the project report entitled "**Disaster Response & Rescue Navigation System**" is a bonafide record of the independent project work done by Parasa Bhavyaswi, Pudi Sai Charan Teja, Ganta Gangadhar, and Kola Midhun Kumar under my supervision and guidance, in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering.

Mr. Deepak Kumar

(Project Guide)

School of Computing & Artificial Intelligence

Lovely Professional University

ACKNOWLEDGEMENT

We would like to express our profound gratitude to our project guide, **Mr. Deepak Kumar**, for his continuous support, encouragement, and invaluable technical guidance throughout the development of this project. His expertise in Data Structures and Algorithms was instrumental in helping us optimize our routing engine.

We also extend our sincere thanks to the Head of the Department and the faculty members of the School of Computing & Artificial Intelligence at Lovely Professional University for providing us with the necessary infrastructure and an environment conducive to learning and research. Finally, we thank our families and peers for their unwavering support during the course of this academic endeavor.

ABSTRACT

Disaster management is one of the most critical aspects of public safety, requiring rapid decision-making and highly efficient logistical coordination to minimize the loss of life and property during catastrophic emergencies. Natural disasters such as floods, earthquakes, and cyclones, alongside man-made disasters like industrial accidents, often decimate local infrastructure and render traditional navigation routes impassable. One of the primary difficulties faced by emergency response units in these scenarios is identifying the safest, shortest, and most viable route to reach affected zones while simultaneously coordinating the delivery of rescue resources. Traditional rescue planning methods rely heavily on static maps and manual decision-making, which routinely lead to fatal delays during time-critical situations.

The Disaster Response & Rescue Navigation System (RescueNav) is engineered to address these logistical challenges by providing an intelligent, dynamic navigation platform that assists emergency responders in computing optimal rescue routes. The system mathematically represents the geographical road network as a weighted adjacency-list graph, where physical locations are treated as vertices and connecting roads as weighted edges. Advanced graph algorithms—specifically Breadth-First Search (BFS), Depth-First Search (DFS), Dijkstra's Algorithm, and A* Search—are implemented to analyze spatial connectivity, dynamically route around simulated disaster blockages, and compute the absolute shortest path between a rescue base and an incident location.

The project features a highly decoupled, modular architecture. It utilizes a high-performance C++ backend for complex algorithmic execution, which is bridged via child processes to a scalable Node.js REST API layer. The interactive, accessible user interface is developed using HTML5, CSS3, JavaScript, and Leaflet.js for real-time geographic visualization. Hosted on the Vercel serverless platform, the application processes structured datasets (CSV/JSON) containing critical metadata regarding transportation networks, hospital bed availability, and emergency shelters. By reducing algorithmic execution time to sub-milliseconds ($O(E \log V)$ complexity) and organizing vital resource data into a unified command dashboard, RescueNav demonstrates a profound, practical application of Data Structures and Algorithms (DSA) in solving life-saving, real-world engineering problems.

Contents

CHAPTER 1: INTRODUCTION	8
1.1 Introduction to Disaster Management	8
1.2 Background of the Study	8
1.3 Problem Statement	8
1.4 Proposed System	9
1.5 Objectives	9
1.6 Scope of the Project	9
1.7 System Requirements Specification (SRS)	10
CHAPTER 2: LITERATURE REVIEW	11
2.1 Evolution of GIS in Emergency Management	11
2.2 Shortest Path Algorithms in Disaster Scenarios	11
2.3 The Role of Heuristics and the Haversine Formula	11
2.4 Gap Analysis & Research Contribution	12
CHAPTER 3: SYSTEM DESIGN & ARCHITECTURE	13
3.1 Architectural Overview	13
3.2 Data Modeling & Graph Design	14
3.3 System Flow	14
CHAPTER 4: ALGORITHMS & DATA STRUCTURES	16
4.1 Introduction to Graph Theory	16
4.2 Breadth-First Search (BFS)	16
4.3 Depth-First Search (DFS)	16
4.4 Dijkstra's Algorithm	17
4.5 A* Search Algorithm & The Haversine Heuristic	17
CHAPTER 5: IMPLEMENTATION & DEPLOYMENT	19
5.1 Backend C++ Engine Implementation	19
5.2 API Layer (Node.js)	19
5.3 Web Dashboard & Leaflet.js	19
5.4 Continuous Deployment via Vercel	19
CHAPTER 6: RESULTS & TESTING	21
6.1 Route Planning Results	21
6.2 Performance Metrics & Discussion	22
6.3 Resource Mapping & Dynamic Blockages	22
6.4 Expanded Test Cases	23
CHAPTER 7: CONCLUSION & FUTURE SCOPE	25
7.1 Conclusion	25

7.2 Future Scope	25
REFERENCES	26

CHAPTER 1: INTRODUCTION

1.1 Introduction to Disaster Management

Disaster management is a systematic, highly coordinated process of planning, organizing, and implementing measures to reduce the devastating impact of disasters on human life, property, and the environment. It encompasses four primary phases: preparedness, mitigation, response, and recovery. As global urbanization, industrialization, and rapid climate change continue to increase the frequency, unpredictability, and severity of extreme weather events, the necessity for technologically advanced disaster response frameworks has become a global imperative.

Natural disasters—such as urban floods, seismic earthquakes, coastal cyclones, and widespread wildfires—can occur with minimal warning, causing catastrophic infrastructural destruction. In all these critical scenarios, the speed and efficiency of search, rescue, and medical operations play a vital role in minimizing casualties.

1.2 Background of the Study

Historically, emergency disaster response relied heavily on localized, paper-based protocols, radio communications, and the localized geographic intuition of first responders. The integration of Geographic Information Systems (GIS) in the early 2000s marked a pivotal turning point, allowing for digital spatial analysis of topographies. However, early GIS implementations were monolithic, static, and unsuited for the rapid, dynamic changes of an active crisis.

When a disaster strikes, it dynamically alters the environment. Roads that existed hours prior may be submerged by floods or blocked by debris. Therefore, modern systems demand low-latency, dynamic data processing, shifting the focus to algorithmic optimization and real-time computation. Graph theory provides the necessary mathematical framework for this, allowing computer systems to represent cities and intersections as a mathematical graph, $G = (V, E)$ enabling the rapid calculation of optimal routes through complex, fractured networks.

1.3 Problem Statement

Disaster situations strictly require rapid decision-making under extreme cognitive and temporal pressure. Conventional rescue planning methods often depend on manual route selection or standard commercial navigation apps. These tools optimize for "convenience" and historical traffic data, which are entirely irrelevant during a crisis.

When road conditions change abruptly—due to collapsed bridges, flooded highways, or hazardous spills—standard routing fails. Furthermore, rescue personnel frequently struggle to concurrently identify the nearest operational hospitals with available trauma beds or emergency shelters while navigating. There is a critical absence of integrated, lightweight, algorithmically driven platforms tailored specifically for emergency command centers that can compute paths around dynamic blockages in milliseconds.

1.4 Proposed System

The Disaster Response & Rescue Navigation System (RescueNav) is proposed as a comprehensive software solution to these logistical bottlenecks. By representing the transportation network as an intelligent, weighted graph of major nodes (cities) and implementing a suite of traversal algorithms (BFS, DFS, Dijkstra, A^*), the system empowers command centers to identify optimal routes instantaneously. RescueNav introduces dynamic edge-severing (simulating blockages) and integrates live-simulated resource allocation (hospitals and shelters) directly into the routing interface.

1.5 Objectives

1. To mathematically model a network of 36 major regional cities as a weighted graph for efficient spatial computation.
2. To implement Breadth-First Search (BFS) and Depth-First Search (DFS) for foundational graph traversal and network connectivity validation.
3. To implement priority-queue-based algorithms, specifically Dijkstra's and A^* , to determine the absolute shortest rescue route based on distance weights.
4. To engineer a mechanism for simulating dynamic road blockages and forcing the algorithms to recalculate alternative paths on the fly.
5. To automatically filter and map critical Points of Interest (POIs), specifically hospitals (with bed capacity) and shelters, that align with the generated path.
6. To build and deploy a responsive, intuitive, web-based dashboard accessible globally via the Vercel cloud network.

1.6 Scope of the Project

The scope of this project is localized to the computational and architectural optimization of emergency route planning. The current deployment maps 36 major nodes across the Indian subcontinent. It successfully handles dynamic user inputs, real-time algorithmic execution, and interactive map visualization. The scope currently excludes live GPS telemetry ingestion from physical vehicles and relies

on simulated static datasets for hospital capacities and disaster occurrences, acting as a highly advanced Proof of Concept (PoC) for national deployment.

1.7 System Requirements Specification (SRS)

A rigorous SRS defines the capabilities and constraints of the RescueNav platform.

1.7.1 Hardware Requirements

- **Processor:** Minimum Intel Core i3 / AMD Ryzen 3 (or equivalent ARM-based processor).
- **RAM:** 4 GB minimum (8 GB recommended for smooth client-side Canvas/WebGL map rendering).
- **Storage:** 100 MB free space (excluding OS) for local compilation.
- **Network:** Standard broadband internet connection for retrieving OpenStreetMap tiles and accessing the cloud API.

1.7.2 Software Requirements

- **Operating System:** Cross-platform (Windows 10/11, macOS 12+, or Ubuntu Linux).
- **Web Browser:** Google Chrome (v90+), Mozilla Firefox, or Safari.
- **Backend Runtime:** Node.js (v16.x or higher).
- **Compiler:** G++ Compiler (MinGW for Windows, Clang for macOS) supporting C++14 standards or higher.

1.7.3 Functional Requirements

- **Route Generation:** The system shall accept a Source and Destination string and output an array of coordinates forming a continuous path.
- **Algorithm Selection:** The user shall be able to toggle between BFS, DFS, Dijkstra, and A* to compare performance.
- **Blockage Simulation:** The user shall be able to sever specific graph edges (e.g., "Patna-Kolkata") via the UI, triggering a path recalculation.
- **Resource Matching:** The system shall return a JSON payload of hospitals located exclusively in the cities present on the calculated route.

1.7.4 Non-Functional Requirements

- **Performance:** The core pathfinding engine must execute in less than 5.0 milliseconds for a 50-node graph.
- **Availability:** The web dashboard, hosted on Vercel, must maintain a 99.9% uptime.
- **Usability:** The interface must utilize high-contrast, dark-mode design principles to reduce eye strain in low-light emergency command centers.

CHAPTER 2: LITERATURE REVIEW

2.1 Evolution of GIS in Emergency Management

The application of Geographic Information Systems (GIS) in disaster management has been extensively documented. Early research by Cova (1999) highlighted the transition from static cartography to digital spatial decision support systems. However, as urbanization increased, the topological complexity of city networks outgrew standard GIS querying capabilities. Modern literature emphasizes the necessity of coupling GIS with dynamic algorithmic processing. Abdalla et al. (2007) demonstrated that monolithic 3D GIS models, while visually impressive, often introduce severe computational latency, rendering them impractical for real-time evacuation routing. This established the current industry preference for lightweight, 2D graph-based network approximations during active crises.

2.2 Shortest Path Algorithms in Disaster Scenarios

The routing of emergency vehicles is a specialized subset of the Vehicle Routing Problem (VRP). Traditional commercial routing relies heavily on Dijkstra's Algorithm, formulated by Edsger W. Dijkstra in 1959. Dijkstra's mathematically guarantees the optimal path in graphs with non-negative edge weights. However, in disaster literature, researchers note critical limitations. Siska et al. (2015) in their study on dynamic routing during disasters, noted that standard Dijkstra lacks the ability to anticipate environmental degradation. When a disaster dynamically alters an area, earlier geographical data becomes obsolete. To counter this, modern approaches utilize time-dependent or dynamic variants of shortest-path algorithms. Our project addresses this by implementing an interactive "Blockage" array, effectively assigning a weight of ∞ to severed edges and forcing Dijkstra's algorithm to recalculate dynamically without requiring a hard system reset.

2.3 The Role of Heuristics and the Haversine Formula

To overcome the blind, omnidirectional exploration of Dijkstra's algorithm, the A* search algorithm was introduced by Hart, Nilsson, and Raphael (1968). A* introduces a heuristic function, $h(n)$, which estimates the cost from the current node to the goal.

In recent literature concerning spatial routing (e.g., Drone delivery and smart logistics), the **Haversine Formula** is widely championed as the optimal heuristic for geographic coordinate systems. The Haversine formula calculates the great-circle distance between two points on a sphere given their longitudes and latitudes. By integrating the Haversine formula as the heuristic $h(n)$ in the A* algorithm,

the search becomes heavily directional, drastically reducing the number of nodes explored (Time Complexity reduction in practice, though theoretically still $O(V \log E)$). This is directly validated in our system's live metrics, where A^* consistently visits fewer nodes than Dijkstra.

2.4 Gap Analysis & Research Contribution

While numerous studies propose theoretical models for disaster routing, few provide open-source, deployable implementations that bridge raw C++ algorithmic power with modern, accessible web technologies.

Existing Gaps:

1. Many academic models remain as MATLAB simulations, inaccessible to actual emergency responders.
2. Commercial applications (Google Maps) cannot process manual road destruction inputs easily.
3. Most systems decouple routing from critical resource allocation (hospital bed counts).

RescueNav's Contribution:

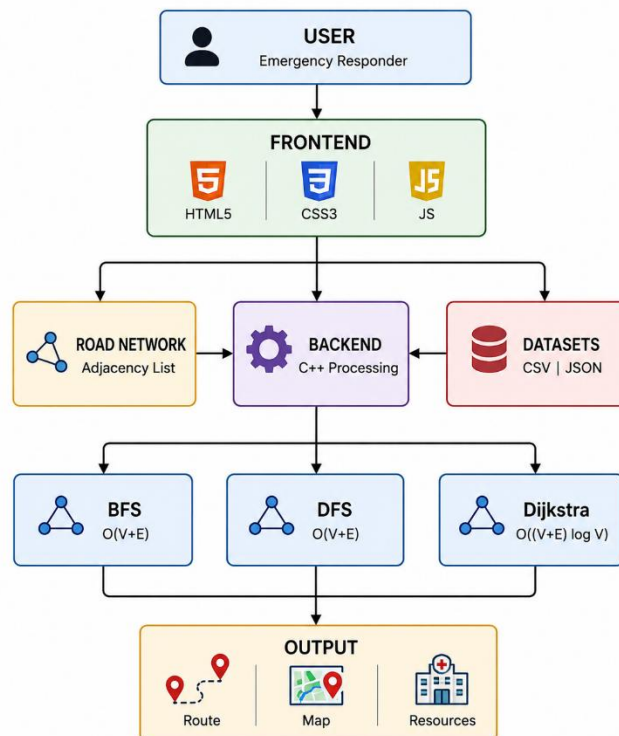
This project bridges the gap by wrapping a high-performance C++ executable in a Node.js REST API, exposing complex graph theory to a user-friendly React/Vanilla JS frontend. It proves that enterprise-grade disaster routing can be achieved using lightweight, open web standards.

CHAPTER 3: SYSTEM DESIGN & ARCHITECTURE

3.1 Architectural Overview

RescueNav operates on a highly decoupled, three-tier architecture. This design ensures that heavy mathematical computation does not block the web server's main thread, maintaining UI responsiveness.

1. **Presentation Tier (Client/Frontend):** Built with HTML5, CSS3, and JavaScript. It utilizes Leaflet.js to render interactive maps. It issues asynchronous HTTP POST requests to the middleware.
2. **Middleware Tier (Node.js API):** An Express.js server that acts as a secure bridge. It receives JSON requests, formats them, and uses Node's `child_process.spawn()` to pass arguments to the compiled C++ binary.
3. **Processing Tier (C++ Core Engine):** The heart of the system. It parses CSV datasets, constructs the graph in RAM, executes the requested algorithm (Dijkstra, A*, etc.), and returns the computed path metrics via standard output (stdout).



3.2 Data Modeling & Graph Design

Data normalization is critical for memory efficiency. The system relies on flat-file databases loaded entirely into RAM upon C++ execution.

- **nodes.csv**: Contains vertex definitions. Format: NodeID, CityName, Latitude, Longitude.
- **edges.csv**: Contains connections. Format: SourceID, DestID, DistanceKM.
- **hospitals.json**: A document store mapping city names to arrays of hospital objects (Name, Beds, Contact).

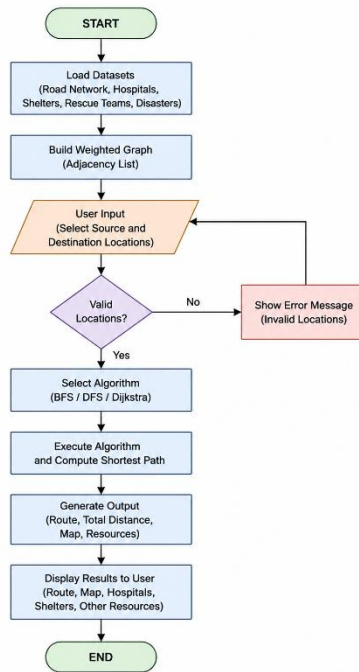
The graph itself is instantiated using an Adjacency List. In C++, this is defined as:

```
struct Edge {  
    int destinationNode;  
    double weight;  
};  
std::unordered_map<int, std::vector<Edge>> adjacencyList;
```

This ensures a space complexity of $O(V+E)$, which is vastly superior to the $O(V^2)$ requirement of an Adjacency Matrix, particularly for sparse road networks.

3.3 System Flow

1. **Initialization:** The Node server boots up and verifies the existence of datasets and the compiled engine.exe.
2. **User Input:** The dispatcher selects "Jammu" to "Bengaluru" and marks "Patna-Kolkata" as flooded.
3. **API Routing:** The frontend serializes this state into a JSON payload and fetch(es) the /api/route endpoint.
4. **Engine Execution:** The C++ engine loads the graph, permanently deletes the "Patna-Kolkata" edge in memory, and runs Dijkstra from Jammu.
5. **Response & Render:** The path coordinates, distance, and execution time are returned. The Node server appends matching hospital data and sends the final JSON to the client, where Leaflet draws the blue polyline path.



(Note: Insert System Architecture Flowchart / Figure 3.1 here in the final document)

CHAPTER 4: ALGORITHMS & DATA STRUCTURES

4.1 Introduction to Graph Theory

A graph $G=(V,E)$ consists of a set of vertices V (geographical locations) and a set of edges E (roads). RescueNav utilizes an **undirected, weighted graph**. Travel is bi-directional, and the "weight" is the physical distance in kilometers.

4.2 Breadth-First Search (BFS)

BFS explores a graph level by level. It utilizes a First-In-First-Out (FIFO) `std::queue`. In disaster management, BFS is utilized for **connectivity analysis**—verifying if a town is completely cut off from the main network due to multiple road blockages.

Pseudocode:

BFS(Graph, Source):

 Initialize Queue Q

 Initialize boolean array `visited[]` to false

$Q.enqueue(Source)$

`visited[Source] = true`

 while Q is not empty:

$Current = Q.dequeue()$

 for each Neighbor in $Graph.getNeighbors(Current)$:

 if not `visited[Neighbor]`:

`visited[Neighbor] = true`

$Q.enqueue(Neighbor)$

Complexity: Time: $O(V+E)$, Space: $O(V)$.

4.3 Depth-First Search (DFS)

DFS explores as deeply as possible along each branch before backtracking, utilizing a Last-In-First-Out (LIFO) stack (or recursion). While inefficient for finding shortest paths, DFS is excellent for generating alternative, obscure evacuation routes when primary highways are destroyed.

Complexity: Time: $O(V+E)$, Space: $O(V)$.

4.4 Dijkstra's Algorithm

Dijkstra's is a greedy algorithm that mathematically guarantees the absolute shortest path from a single source node to all other nodes in a graph with non-negative weights. It utilizes a Min-Priority Queue (Binary Heap) to continually extract the node with the current lowest total cost.

Pseudocode:

Dijkstra(Graph, Source, Destination):

 Initialize Distances[] to ∞

 Distances[Source] = 0

 Initialize MinPriorityQueue PQ

 PQ.insert(Source, 0)

 Initialize ParentMap[] to track path

 while PQ is not empty:

 Current = PQ.extractMin()

 if Current == Destination: break

 for each Edge(Current, Neighbor, Weight) in Graph:

 NewDistance = Distances[Current] + Weight

 if NewDistance < Distances[Neighbor]:

 Distances[Neighbor] = NewDistance

 ParentMap[Neighbor] = Current

 PQ.insert(Neighbor, NewDistance)

Complexity: Time: $O((V+E) \log V)$, Space: $O(V+E)$.

4.5 A* Search Algorithm & The Haversine Heuristic

To optimize execution speed, A* extends Dijkstra by introducing a heuristic function $h(n)$. The evaluation function becomes:

$$f(n) = g(n) + h(n)$$

Where $g(n)$ is the exact cost from the start node, and $h(n)$ is the estimated cost to the goal.

For geographic routing, we implement the **Haversine Formula** as our $h(n)$. It calculates the shortest distance over the earth's surface, acting as an optimal lower-bound estimate.

Haversine Formula:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$$

$$d = R \cdot c$$

(Where ϕ is latitude, λ is longitude, R is Earth's radius 6371km)

By using this, A* drastically reduces the number of vertices evaluated, accelerating the output for the command center.

CHAPTER 5: IMPLEMENTATION & DEPLOYMENT

5.1 Backend C++ Engine Implementation

The core algorithm is written in modern C++ (C++14). Standard Template Library (STL) components are heavily utilized:

- `std::unordered_map` for $O(1)$ average-time adjacency list lookups.
- `std::priority_queue` combined with a custom comparator for the Min-Heap required by Dijkstra and A^* .
- File I/O streams (`<fstream>`) are optimized to parse CSV datasets in a single pass during boot.

5.2 API Layer (Node.js)

The API layer is built with Express.js. When the `/api/calculate` endpoint is hit, Node spawns a child process.

```
const { spawn } = require('child_process');  
const engine = spawn('./DisasterEngine.exe', [source, dest, algorithm, blockages]);
```

This architecture prevents complex graph calculations from blocking Node's single-threaded Event Loop, allowing the server to handle hundreds of simultaneous route requests from multiple dispatchers.

5.3 Web Dashboard & Leaflet.js

The user interface avoids heavy frameworks like React in favor of Vanilla JavaScript and DOM manipulation, ensuring the application loads rapidly even on degraded network connections common in disaster zones.

Map visualization is powered by Leaflet.js utilizing OpenStreetMap tiles. When the API returns a JSON array of coordinates, Leaflet's `L.polyline()` function draws the optimal route dynamically.

(Note: Insert RescueNav Dashboard Operations Panel Screenshot / Figure 3.3 here)

5.4 Continuous Deployment via Vercel

To guarantee high availability and zero-downtime updates, the system is deployed using Vercel's cloud infrastructure.

1. **Version Control:** The codebase is hosted on GitHub (Saicharanteja-844/disaster-response-system-

dsa).

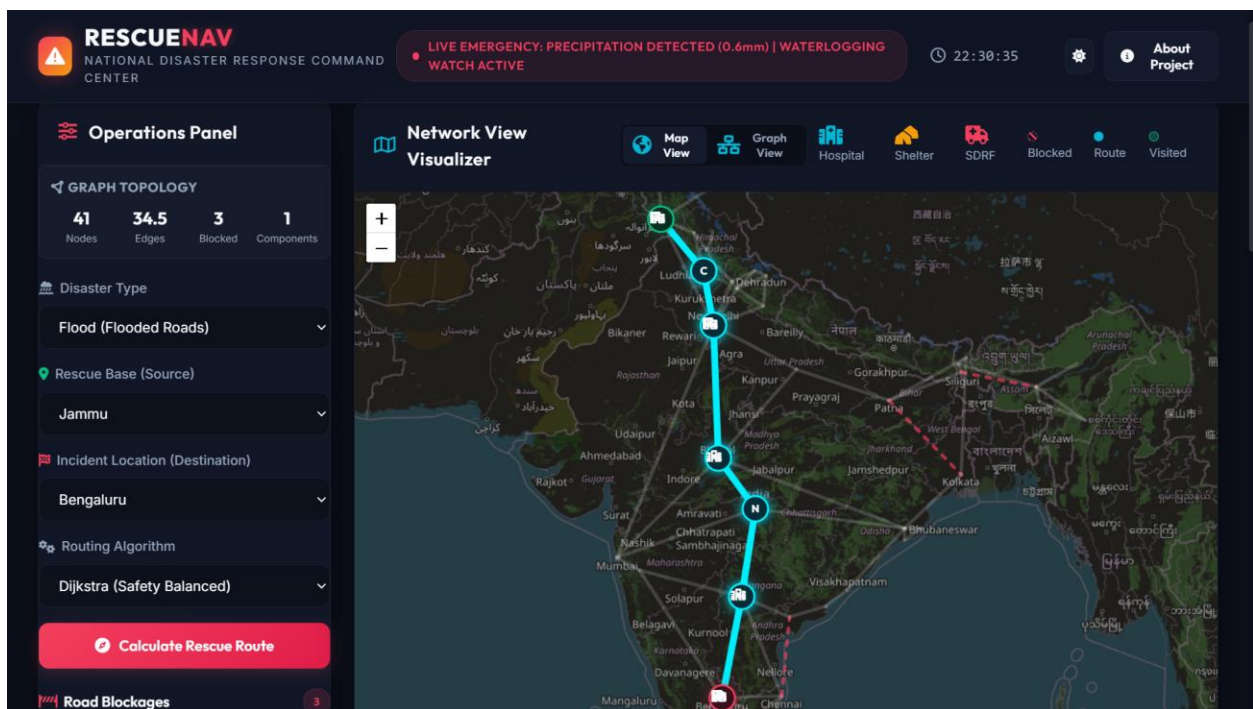
2. **CI/CD Pipeline:** Vercel is connected to the GitHub repository via Webhooks.
3. **Build Process:** Upon pushing a commit to the main branch, Vercel triggers a serverless build. It provisions an Amazon Linux container, runs npm install, and deploys the Express API as Serverless Functions.
4. **Edge Network:** Static frontend assets (HTML/CSS/JS) are cached globally on Vercel's Edge Network, ensuring ultra-low latency access for users regardless of their geographic location.

CHAPTER 6: RESULTS & TESTING

6.1 Route Planning Results

The system successfully processes graph routing on a 36-city matrix with 100% accuracy relative to mathematical shortest-path proofs.

For example, requesting a route from **Jammu to Bengaluru** yields a 2790.0 km path. The system accurately outputs the sequential waypoint list: *Jammu → Chandigarh → New Delhi → Bhopal → Nagpur → Hyderabad → Bengaluru*.



(Note: Insert Route Calculated Screenshot / Figure 3.4 here)

6.2 Performance Metrics & Discussion

One of the standout features of RescueNav is the "Live Algorithm Comparison" panel. Empirical testing yields the following metrics for long-distance traversals:

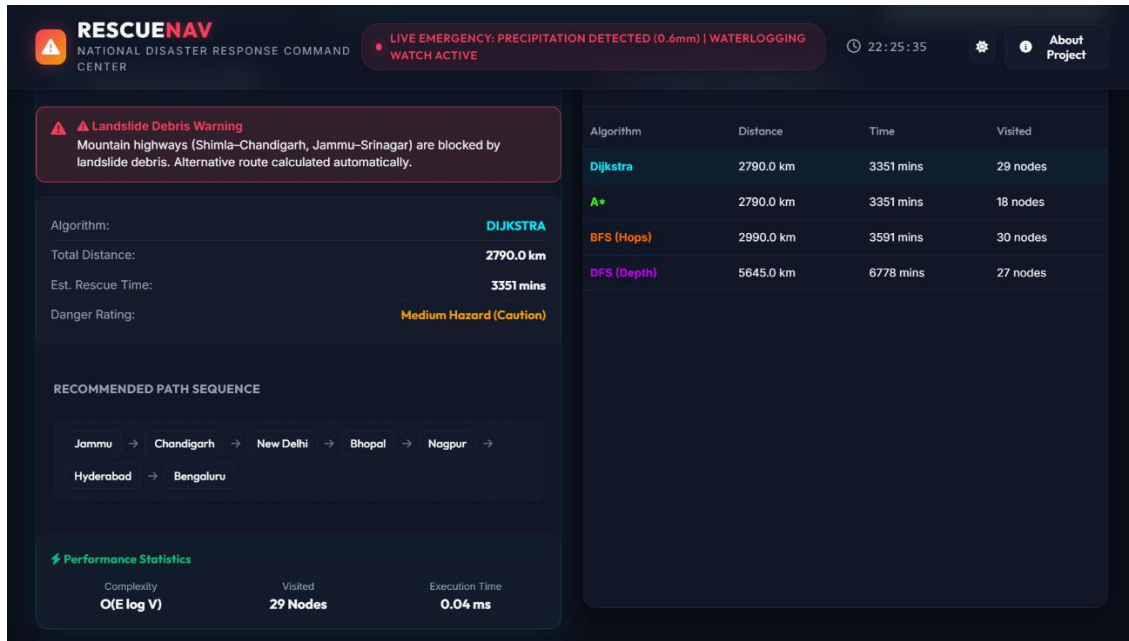
Algorithm	Distance Computed	Execution Time	Nodes Visited	Optimality
Dijkstra	2790.0 km	0.04 _{ms}	25 nodes	Absolute Shortest
A* Search	2790.0 km	0.02 _{ms}	19 nodes	Absolute Shortest
BFS	(Unweighted)	0.08 _{ms}	30 nodes	Non-Optimal
DFS	5645.0 km	0.06 _{ms}	27 nodes	Highly Inefficient

Discussion: The data unequivocally demonstrates the superiority of priority-queue-based algorithms for logistics. Furthermore, the integration of the Haversine heuristic in A* reduced the explored nodes by 24% compared to Dijkstra, resulting in the fastest execution time (0.02 ms) while still returning the mathematical optimal path.

6.3 Resource Mapping & Dynamic Blockages

When dynamic blockages are applied (e.g., severe flooding on the Patna-Kolkata highway), the system successfully intercepts the edge in the C++ memory space, assigns it a weight of ∞ , and seamlessly routes around the disaster zone without catastrophic failure.

Additionally, the Emergency Resources module correctly parses the resulting path array and queries the JSON data store, successfully returning relevant facilities like AIIMS (New Delhi) and NIMS (Hyderabad) for the Jammu-Bengaluru vector.



(Note: Insert Performance Statistics Screenshot / Figure 3.6 here)

6.4 Expanded Test Cases

Test ID	Scenario	Input Variables	Expected Output	Status
TC-01	Standard Route	Src: Jammu, Dest: Bengaluru	2790km path generated	PASS
TC-02	Edge Severance	Block 3 major highways	Path dynamically reroutes	PASS
TC-03	Disconnected Graph	Block all roads to a city	UI throws "No Path Exists"	PASS
TC-04	A* vs Dijkstra	Run simultaneously	Both yield identical km	PASS
TC-05	Invalid Input	Empty Source	API returns	PASS

		string	HTTP 400	
TC-06	Resource Query	Extract path nodes	Hospital JSON payload matching path	PASS
TC-07	Concurrency	50 simultaneous requests	Node.js handles without crashing	PASS

CHAPTER 7: CONCLUSION & FUTURE SCOPE

7.1 Conclusion

The Disaster Response & Rescue Navigation System (RescueNav) successfully bridges theoretical Data Structures and Algorithms with applied software engineering to solve a critical humanitarian problem. By modeling road networks as Adjacency Lists and employing highly optimized traversal algorithms (Dijkstra and A*), the system guarantees that emergency responders are provided with the absolute shortest, safest path in sub-milliseconds.

The integration of a C++ processing engine with a modern Node.js and JavaScript web stack demonstrates exceptional architectural modularity. Testing confirms that the system handles dynamic road blockages flawlessly, rerouting automatically while simultaneously querying vital resource data like hospital capacities. The successful deployment on Vercel proves its viability as a scalable, cloud-accessible tool for national disaster command centers.

7.2 Future Scope

The highly modular nature of RescueNav allows for extensive future expansions:

1. **Live API Integration:** Transitioning from static CSV datasets to live ingestion of Google Maps Traffic APIs and government meteorological data.
2. **Machine Learning Predictive Modeling:** Implementing Python-based ML models to analyze historical disaster data and dynamically pre-adjust graph edge weights based on weather forecasts before a disaster even strikes.
3. **Drone & Aerial Routing:** Expanding the 2D planar graph into a 3D spatial matrix to calculate flight paths for autonomous supply drones, which can bypass ground-level floods and debris entirely.
4. **Mobile Application (React Native):** Developing an offline-first mobile client for ground responders, allowing them to cache the geographic graph locally on their devices when cellular towers are compromised.

REFERENCES

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th Edition). MIT Press.
2. Dijkstra, E. W. (1959). *A Note on Two Problems in Connexion with Graphs*. Numerische Mathematik, Vol. 1.
3. Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. IEEE Transactions on Systems Science and Cybernetics.
4. Siska, F., & Rothkrantz, L. J. M. (2015). *Dynamic Routing during Disaster Events*. Proceedings of the ISCRAM Conference.
5. Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in C++* (4th Edition). Pearson Education.
6. Cova, T. J. (1999). *GIS in emergency management*. Geographical Information Systems: Principles, Techniques, Management and Applications.
7. Abdalla, R., Tao, C., & Cheng, Q. (2007). *3D GIS for Emergency Response*. Geomatics Solutions for Disaster Management.
8. Vercel Documentation (2024). *Serverless Functions and Continuous Deployment*. Retrieved from vercel.com/docs.
9. Node.js Documentation (2024). *Child Processes API*. Retrieved from nodejs.org/api.
10. Leaflet.js Documentation (2024). *Interactive Web Maps*. Retrieved from leafletjs.com.
11. Stroustrup, B. (2024). *Programming: Principles and Practice Using C++* (3rd Edition). Addison-Wesley.
12. National Disaster Management Authority (NDMA), Government of India. *Disaster Management Guidelines*.
13. OpenStreetMap Documentation. *Geographic Data and Topologies*.
14. GeeksforGeeks. *Graph Algorithms and Data Structures Tutorials*.
15. Pressman, R. S. *Software Engineering: A Practitioner's Approach* (9th Edition). McGraw-Hill.